



COMP3153/9153

Algorithmic Verification

Lecture 13: Timed Automata and Languages

Timed Systems

The systems we have examined so far have a notion of time, but only of events happening one after the other. This is an abstraction called *discrete time*.

Timed Systems

The systems we have examined so far have a notion of time, but only of events happening one after the other. This is an abstraction called *discrete time*.

For *dense time* systems, we care about a real-valued *continuous* time clocks.

Timed Systems

The systems we have examined so far have a notion of time, but only of events happening one after the other. This is an abstraction called *discrete time*.

For *dense time* systems, we care about a real-valued *continuous* time clocks.

Example (Dense Time System)

A light controlled by one button, where a “double press” of the button increases the brightness of the light. The second button press must be at most 3 time units after the first button press for the “double press” behaviour to trigger.

After 12 time units, the light must turn off.

Can we get away with discrete time?

No

Can we get away with discrete time?

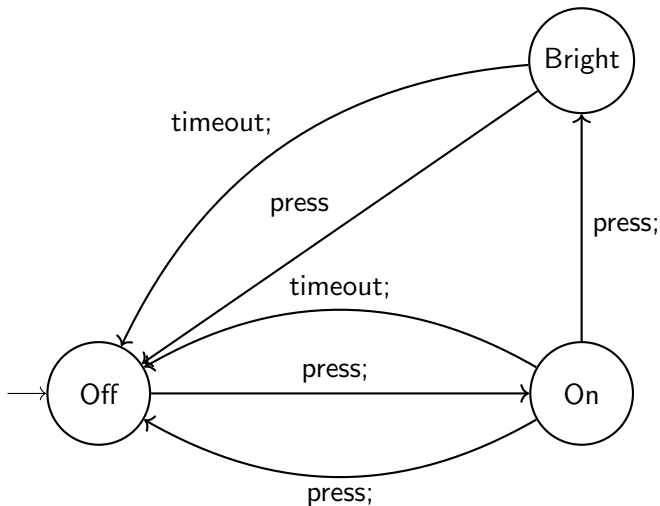
No

Theorem (Brzozowski and Seger)

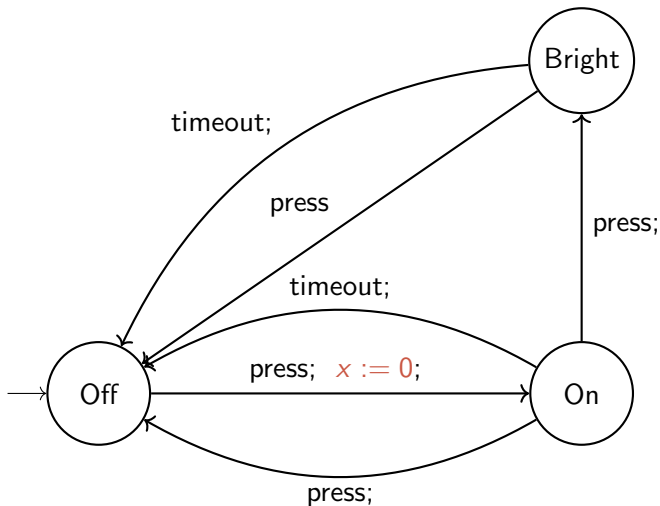
For every $k \geq 1$ there is a system where the set of states reachable in dense time is **strictly larger** than the set of states reachable in discrete time in $\frac{1}{k}$ steps.

This is shown for asynchronous circuits, but applies generally.

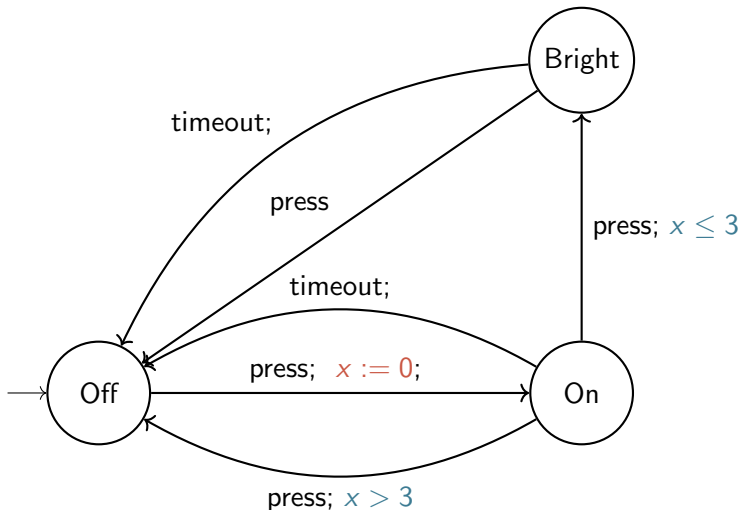
Towards Timed Automata



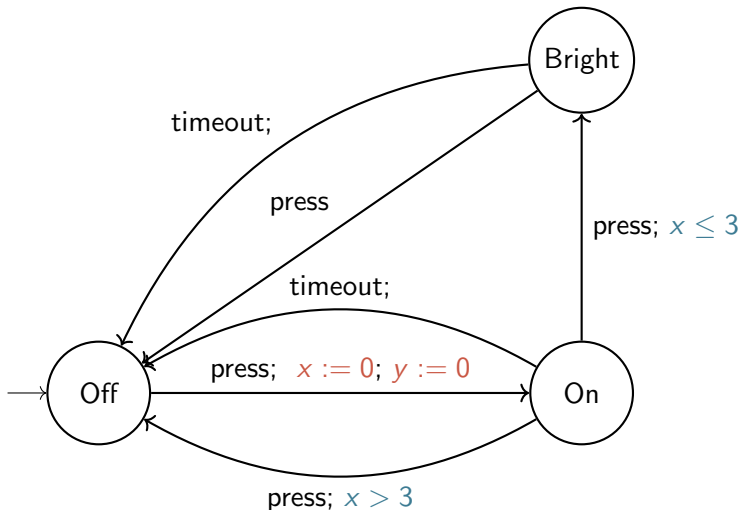
Towards Timed Automata



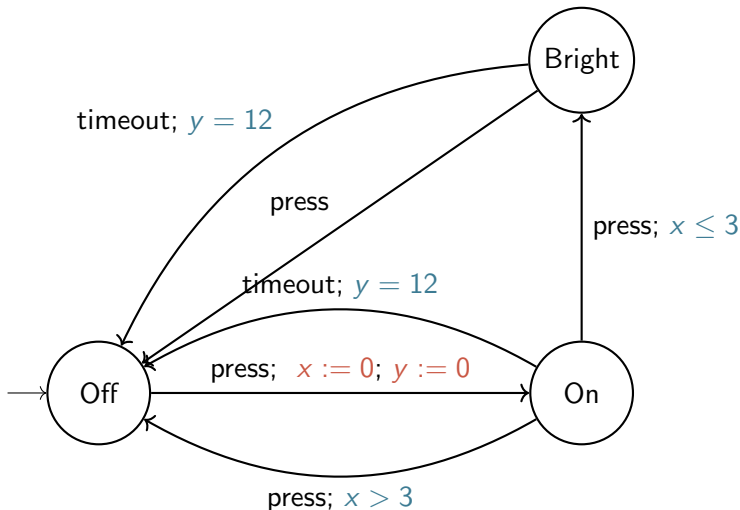
Towards Timed Automata



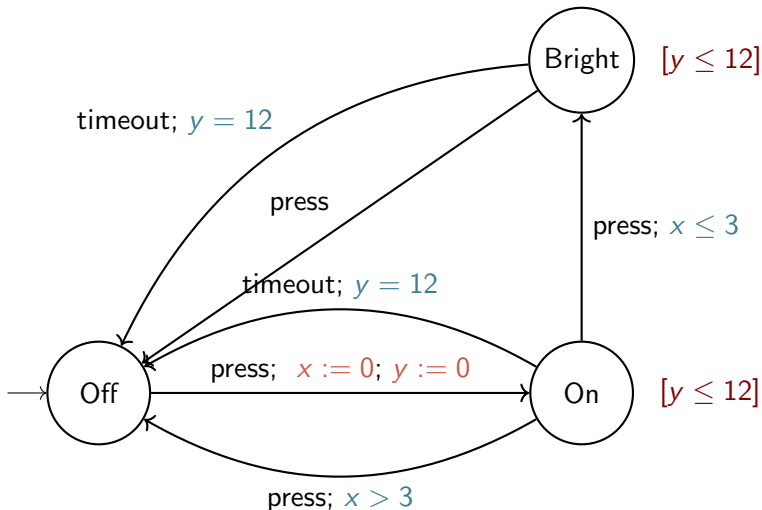
Towards Timed Automata



Towards Timed Automata



Towards Timed Automata



Timed Automata

A timed automaton is a finite automaton with one or more real valued clocks. Transitions are annotated with **resets** and **guards** and states may be annotated with **invariants**.

Timed Automata

A timed automaton is a finite automaton with one or more real valued clocks. Transitions are annotated with **resets** and **guards** and states may be annotated with **invariants**.

Definition

A timed automaton $\mathcal{A}$ is a 6-tuple $(L, \ell_0, \text{Act}, X, \text{Inv}, \longrightarrow)$ where:

- L is a set of locations.
- ℓ_0 is the initial location.
- Act is the set of discrete actions.
- X is the set of clock variables.
- $\text{Inv}(\ell)$ is a **clock constraint** invariant associated with ℓ .
- Transitions are defined as $\ell \xrightarrow{g;a;r} \ell'$ where
 - g is zero or more clock constraint **guards**.
 - a is an action $\in \text{Act}$
 - r is zero or more **clock resets**

Clock Constraints

For reasons that will become clear later, we want to restrict clock constraints to linear subtractions:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

Example (For the light automaton...)

(Off, $x = y = 0$)

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

Example (For the light automaton...)

$$(\text{Off}, x = y = 0) \xrightarrow{265 + \pi^2} (\text{Off}, x = y = 265 + \pi^2)$$

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

Example (For the light automaton...)

$$\begin{aligned} (\text{Off}, x = y = 0) &\xrightarrow{265 + \pi^2} (\text{Off}, x = y = 265 + \pi^2) \xrightarrow{\text{press}} \\ &(\text{On}, x = y = 0) \end{aligned}$$

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

Example (For the light automaton...)

$$\begin{aligned} (\text{Off}, x = y = 0) &\xrightarrow{265 + \pi^2} (\text{Off}, x = y = 265 + \pi^2) \xrightarrow{\text{press}} \\ &(\text{On}, x = y = 0) \xrightarrow{3.2} (\text{On}, x = y = 3.2) \end{aligned}$$

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

Example (For the light automaton...)

$$\begin{aligned} (\text{Off}, x = y = 0) &\xrightarrow{265 + \pi^2} (\text{Off}, x = y = 265 + \pi^2) \xrightarrow{\text{press}} \\ &(\text{On}, x = y = 0) \xrightarrow{3.2} (\text{On}, x = y = 3.2) \xrightarrow{\text{press}} \\ &(\text{Off}, x = y = 3.2) \end{aligned}$$

States and Runs

States

The *state* of a timed automaton is a tuple of the location $\in L$ and the values of all clocks $\in \mathbb{R}$.

Runs

A *run* of a timed automaton is some interleaving of *delay* steps (which pass some time) and *discrete* steps, which take an action.

Example (For the light automaton...)

$$\begin{aligned} (\text{Off}, x = y = 0) &\xrightarrow{265 + \pi^2} (\text{Off}, x = y = 265 + \pi^2) \xrightarrow{\text{press}} \\ &(\text{On}, x = y = 0) \xrightarrow{3.2} (\text{On}, x = y = 3.2) \xrightarrow{\text{press}} \\ (\text{Off}, x = y = 3.2) &\xrightarrow{27.87} (\text{Off}, x = y = 31.07) \longrightarrow \dots \end{aligned}$$

Product

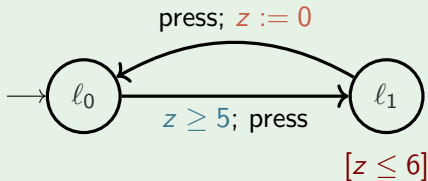
The product of timed automata is as the discrete time product, where we take the conjunction of all guards and invariants, and the union of all resets.

Product

The product of timed automata is as the discrete time product, where we take the conjunction of all guards and invariants, and the union of all resets.

Example (Boardwork)

Let's compute the product of the light automaton with this user automaton:



Timed Words and Languages

Timed Words

A timed word is a finite or infinite sequence of pairs of actions and time stamps.

Timed Words and Languages

Timed Words

A timed word is a finite or infinite sequence of pairs of actions and time stamps.

Example (overlines indicate sequences)

$$L_1 = \{\overline{(a, t)} \mid \bar{a} = (01)^*0 \wedge \forall i \geq 0. t_{2(i+1)} - t_{2i} = 3\}$$

$$L_2 = \{\overline{(a, t)} \mid \bar{a} = (01)^\omega \wedge \forall i \geq 0. t_{2i+1} \leq t_{2i} + 1\}$$

Timed Words and Languages

Timed Words

A timed word is a finite or infinite sequence of pairs of actions and time stamps.

Example (overlines indicate sequences)

$$L_1 = \{\overline{(a, t)} \mid \bar{a} = (01)^*0 \wedge \forall i \geq 0. t_{2(i+1)} - t_{2i} = 3\}$$
$$L_2 = \{\overline{(a, t)} \mid \bar{a} = (01)^\omega \wedge \forall i \geq 0. t_{2i+1} \leq t_{2i} + 1\}$$

Timed Languages

Extend TA definition of $\mathcal{A}$ with a set F of **final states** and a set R of **repeating states**.

- A finite word w is $\in \mathcal{L}(\mathcal{A})$ iff a run generating the word w ends in a state F .
- An infinite word w is $\in \mathcal{L}(\mathcal{A})$ iff a run generating the word w visits states in R infinitely often.

Timed Regular Languages

Definition

A language is *timed regular* iff there exists a timed automaton that recognises it.

Timed Regular Languages

Definition

A language is *timed regular* iff there exists a timed automaton that recognises it.

Timed regular languages are closed under union and intersection in the usual way — merge the initial states and product respectively.

Timed Regular Languages

Definition

A language is *timed regular* iff there exists a timed automaton that recognises it.

Timed regular languages are closed under union and intersection in the usual way — merge the initial states and product respectively.

Problem

Write a timed automaton for words on the alphabet $\{a, b\}$ that contain two a 's exactly 1 time unit apart.

Timed Regular Languages

Definition

A language is *timed regular* iff there exists a timed automaton that recognises it.

Timed regular languages are closed under union and intersection in the usual way — merge the initial states and product respectively.

Problem

Write a timed automaton for words on the alphabet $\{a, b\}$ that contain two a 's exactly 1 time unit apart.

Complement is not timed regular $\Rightarrow$ not closed.

ε -Transitions

Unlike for discrete time systems, ε transitions add power for timed automata.

ε -Transitions

Unlike for discrete time systems, ε transitions add power for timed automata.

Example

Consider the language where actions must occur on integer time stamps. This can be done with a ε reset, but cannot be expressed as a timed automaton without ε .

True Invariants

Do we need invariants?

True Invariants

Do we need invariants? Not to recognise a given timed regular language.

This is because our timed words only pair time stamps with discrete actions, so violating invariants by sitting still does not change the set of recognised words.

True Invariants

Do we need invariants? Not to recognise a given timed regular language.

This is because our timed words only pair time stamps with discrete actions, so violating invariants by sitting still does not change the set of recognised words.

So, we just move the invariants to both the incoming and outgoing transitions like so:

$$\ell_1 \xrightarrow{g;a;r} \ell_2$$

becomes

$$\ell_1 \xrightarrow{g \wedge \text{Inv}(\ell_1) \wedge r(\text{Inv}(\ell_2));a;r} \ell_2$$

Where $r(\varphi)$ is applying the resets r as a substitution to φ .

Reachability

The state space of timed automata are **uncountably infinite** (because of real valued clocks).

Reachability

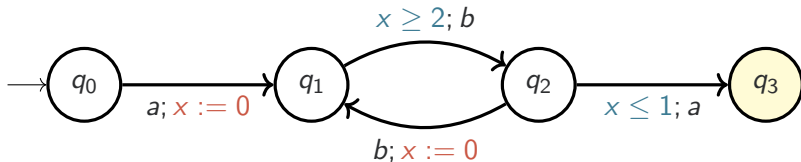
The state space of timed automata are **uncountably infinite** (because of real valued clocks).

How can we compute if a state is reachable?

Reachability

The state space of timed automata are **uncountably infinite** (because of real valued clocks).

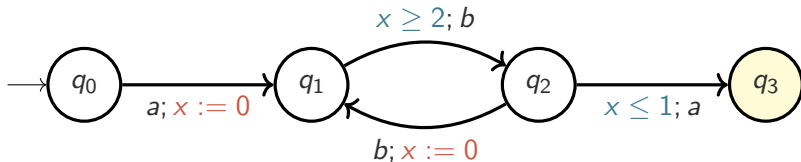
How can we compute if a state is reachable?



Reachability

The state space of timed automata are **uncountably infinite** (because of real valued clocks).

How can we compute if a state is reachable?



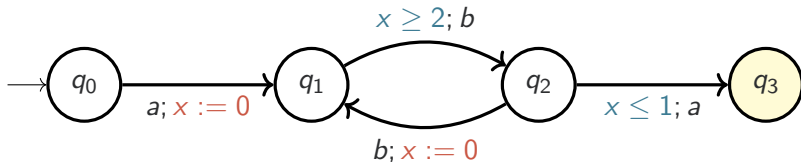
Key Idea

We will group clock values into **equivalence classes** to make a **finite abstraction** of the timed automata which **preserves reachability**.

Reachability

The state space of timed automata are **uncountably infinite** (because of real valued clocks).

How can we compute if a state is reachable?



Key Idea

We will group clock values into **equivalence classes** to make a **finite abstraction** of the timed automata which **preserves reachability**.

Instead of specific amounts of time passing, we will add a generic “time passing” action called δ to our abstraction.

Time Abstraction

Ignoring the specific time elapsed and instead just using δ is called *time abstraction*.

Time Abstraction

Ignoring the specific time elapsed and instead just using δ is called *time abstraction*.

Equivalences

Our normal notions of bisimulation and trace equivalence apply also to timed automata. For example, two timed automata are *timed trace equivalent* iff they accept the same *timed language*.

Two automata are *time-abstract equivalent* iff they appear equivalent after applying time abstraction.

Time Abstraction

Ignoring the specific time elapsed and instead just using δ is called *time abstraction*.

Equivalences

Our normal notions of bisimulation and trace equivalence apply also to timed automata. For example, two timed automata are *timed trace equivalent* iff they accept the same *timed language*.

Two automata are *time-abstract equivalent* iff they appear equivalent after applying time abstraction.

Timed equivalence is *finer* than time-abstract equivalence, but time-abstract systems are more tractable.

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

- 1 Integer Points: $x = 0 \wedge y = 0$? **Yep!**
- 2 Open Intervals: $x = 0 \wedge 2 < y < 3$? **Yep!**

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

- 1 Integer Points: $x = 0 \wedge y = 0$? **Yep!**
- 2 Open Intervals: $x = 0 \wedge 2 < y < 3$? **Yep!**
- 3 Open Squares: $0 < x < 1 \wedge 1 < y < 2$?

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

- 1 Integer Points: $x = 0 \wedge y = 0$? **Yep!**
- 2 Open Intervals: $x = 0 \wedge 2 < y < 3$? **Yep!**
- 3 Open Squares: $0 < x < 1 \wedge 1 < y < 2$? **No! We can go finer!**

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

- 1 Integer Points: $x = 0 \wedge y = 0$? **Yep!**
- 2 Open Intervals: $x = 0 \wedge 2 < y < 3$? **Yep!**
- 3 Open Squares: $0 < x < 1 \wedge 1 < y < 2$? **No! We can go finer!**
- 4 Diagonals: $x - y = 1 \wedge 1 < x < 2$?

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

- 1 Integer Points: $x = 0 \wedge y = 0$? **Yep!**
- 2 Open Intervals: $x = 0 \wedge 2 < y < 3$? **Yep!**
- 3 Open Squares: $0 < x < 1 \wedge 1 < y < 2$? **No! We can go finer!**
- 4 Diagonals: $x - y = 1 \wedge 1 < x < 2$? **Yep!**
- 5 Open Triangles: $x - y < 2 \wedge 3 < x < 4$?

Regions

Regions

Our clock valuations are grouped into *regions*. Regions should:

- 1 Satisfy the same clock constraints.
- 2 Reach the same regions from time passing (called the *successor regions*).

Recall our definition of clock constraints:

$$\varphi ::= x \sim k \mid x - y \sim k \mid \varphi_1 \wedge \varphi_2$$

where $x, y \in X$ and $k \in \mathbb{Z}$ and $(\sim) \in \{<, \leq, =, \geq, >\}$

The smallest regions these constraints can distinguish (in 2D):

- 1 Integer Points: $x = 0 \wedge y = 0$? **Yep!**
- 2 Open Intervals: $x = 0 \wedge 2 < y < 3$? **Yep!**
- 3 Open Squares: $0 < x < 1 \wedge 1 < y < 2$? **No! We can go finer!**
- 4 Diagonals: $x - y = 1 \wedge 1 < x < 2$? **Yep!**
- 5 Open Triangles: $x - y < 2 \wedge 3 < x < 4$? **Yep!**

A finite bound

Depending on the specific constraints involved, we can sometimes merge regions where the difference between them is not important.

A finite bound

Depending on the specific constraints involved, we can sometimes merge regions where the difference between them is not important.

Maximal Constant

The *maximal constant* K of a system is the highest integer constant that occurs in the set of all clock constraints. Regions dealing with clock values between K and ∞ can always be merged.

A finite bound

Depending on the specific constraints involved, we can sometimes merge regions where the difference between them is not important.

Maximal Constant

The *maximal constant* K of a system is the highest integer constant that occurs in the set of all clock constraints. Regions dealing with clock values between K and ∞ can always be merged.

The maximal constant merging gives us a finite bound on the number of regions: $\mathcal{O}(|X|! \cdot K^{|X|})$

Region Graph Definition

Region Graph

The region graph of a TA $\mathcal{A} = (L, \ell_0, \text{Act}, X, \text{Inv}, \longrightarrow)$ is the automaton $(L \times \text{Regions}(\mathcal{A}), (\ell_0, \bar{0}), \text{Act} \cup \{\delta\}, \Longrightarrow)$ where:

- $\text{Regions}(\mathcal{A})$ is the set of all regions.
- $\bar{0}$ is the region where all clock values are zero.

Region Graph Definition

Region Graph

The region graph of a TA $\mathcal{A} = (L, \ell_0, \text{Act}, X, \text{Inv}, \longrightarrow)$ is the automaton $(L \times \text{Regions}(\mathcal{A}), (\ell_0, \bar{0}), \text{Act} \cup \{\delta\}, \Longrightarrow)$ where:

- $\text{Regions}(\mathcal{A})$ is the set of all regions.
- $\bar{0}$ is the region where all clock values are zero.
- For $a \in \text{Act}$, $(\ell, R) \xRightarrow{a} (\ell', R')$ if
 - There is an edge $\ell \xrightarrow{g;a;r} \ell'$,
 - R implies g
 - $R' = r(R)$

Region Graph Definition

Region Graph

The region graph of a TA $\mathcal{A} = (L, \ell_0, \text{Act}, X, \text{Inv}, \longrightarrow)$ is the automaton $(L \times \text{Regions}(\mathcal{A}), (\ell_0, \bar{0}), \text{Act} \cup \{\delta\}, \Longrightarrow)$ where:

- $\text{Regions}(\mathcal{A})$ is the set of all regions.
- $\bar{0}$ is the region where all clock values are zero.
- For $a \in \text{Act}$, $(\ell, R) \xRightarrow{a} (\ell', R')$ if
 - There is an edge $\ell \xrightarrow{g;a;r} \ell'$,
 - R implies g
 - $R' = r(R)$
- $(\ell, R) \xRightarrow{\delta} (\ell', R')$ if
 - R' is a successor of R
 - R implies $\text{Inv}(\ell)$ and R' implies $\text{Inv}(\ell')$

Region Graph Definition

Region Graph

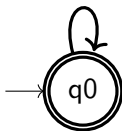
The region graph of a TA $\mathcal{A} = (L, \ell_0, \text{Act}, X, \text{Inv}, \longrightarrow)$ is the automaton $(L \times \text{Regions}(\mathcal{A}), (\ell_0, \bar{0}), \text{Act} \cup \{\delta\}, \Longrightarrow)$ where:

- $\text{Regions}(\mathcal{A})$ is the set of all regions.
- $\bar{0}$ is the region where all clock values are zero.
- For $a \in \text{Act}$, $(\ell, R) \xRightarrow{a} (\ell', R')$ if
 - There is an edge $\ell \xrightarrow{g;a;r} \ell'$,
 - R implies g
 - $R' = r(R)$
- $(\ell, R) \xRightarrow{\delta} (\ell', R')$ if
 - R' is a successor of R
 - R implies $\text{Inv}(\ell)$ and R' implies $\text{Inv}(\ell')$
- For any **open region** R we also have $(\ell, R) \xRightarrow{\delta} (\ell, R)$.

Tiny Example

Let's try a simple region graph following the formal definition for this one-dimensional, one state TA:

$x \geq 2; a; x := 0$



Key Properties

The big win

A location is reachable in the region graph of a timed automaton A iff it is reachable in A .

Key Properties

The big win

A location is reachable in the region graph of a timed automaton A iff it is reachable in A .

As the region graph is just a normal finite automaton, reachability is **decidable** for TA, although PSPACE-complete.

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/sCi2xjavrS>